Natural Language Processing Applications

Dr. Chetana Gavankar, Ph.D,
IIT Bombay-Monash University Australia
Chetana.gavankar@pilani.bits-pilani.ac.in

BITS Pilani
Pilani Campus

Session Content

- QA versus Conversational AI
- QA Systems Journey
- Current Approaches
  - Retrieval Based QA
  - Knowledge bases QA and Hybrid QA
  - Extractive QA – DL approach
  - Generative QA
  - RAG
  - Agentic AI QA
- Case study
- Evaluation Measures

What is Question Answering

A field of Natural Language Processing (NLP) focused on building systems that can automatically answer questions posed by humans in natural language.

The Goal: Move beyond simple keyword search to provide a **direct, accurate, and concise answer** to a user's query.

- **QA:** You ask "what temperature to bake bread," and it answers, "**350°F (175°C).**"

Why is QA So Important? (The Business Value)

Customer Support: Instantly answer 80% of repetitive customer questions 24/7.

Enterprise Knowledge: Allow employees to "ask questions" of your company's internal documents (HR policies, technical manuals, sales data).

Data Analysis: Enable executives to ask, "What was our top-selling product in Q3?" and get a direct answer, not a complex dashboard.

User Experience: Powers the voice assistants and search engines we use every day (Siri, Alexa, Google).

QA versus Conversational AI

Feature	Question Answering (QA)	Conversational AI
Core Goal	1. **Answer** a single question.	1. **Hold** a multi-turn dialogue.
Scope	2. **Transactional** (one-shot).	2. **Relational** and **Goal-Oriented**.
Context	3. **Stateless** (forgets immediately).	3. **Stateful** (must remember conversation history).
Example	**You:** "What is the capital of France?" **Bot:** "Paris."	**You:** "I need to book a flight." **Bot:** "Sure, where are you flying to?" **You:** "Paris." **Bot:** "Great. When do you want to leave for Paris?"

When should we use conversational AI for QA

Conversational AI can absolutely perform QA, but you should only use it when the QA task is complex enough to benefit from:

- reasoning
- multi-turn context
- unstructured text understanding
- flexibility

For simple, repetitive questions, **conversational AI is overkill**—a simple QA system is cheaper, faster, safer.

QA Real world applications

innovate achieve lead

1. Questions are predictable or repetitive

Example: “What are your store hours?”, “What is the refund policy?”

2. Answers must be fixed and controlled

Example: Banking compliance messages, Medical safety instructions

3. You need very fast responses with low cost

Example: An FAQ chatbot handling thousands of queries per minute

4. High accuracy is required and hallucinations are risky

Example: Legal policies, insurance rules, company procedures

5. Single-turn interactions (no context needed)

Example: “What is the capital of Japan?”

6. Information is pulled directly from a database or documents

Example: “Show me my order status.”, “What is CPU usage right now?”

Conversational AI Real world applications

innovate achieve lead

1. **Multi-turn conversations**

Example: User: “Book a flight.”
Bot: “Where to?”
User: “London.”

2. Questions require reasoning or explanation

Example: “Explain how transformers differ from SSMs.”, “Summarize this report.”

3. The question depends on previous context

Example: User: “Who is ?”, “How old is he?”

4. Handling unclear or ambiguous questions

Example: “Tell me about delivery charges.” (Bot: “Domestic or international?”)

5. Working with unstructured documents (PDFs, long texts)

Example: “What are the main points of this 30-page contract?”

6. Conversational tasks beyond QA

Example: recommending products, tutoring, guiding, troubleshooting.

Apple's Siri


AT&T
22:26
65 %
“ Do I need an umbrella tomorrow in San Francisco ”
Yes, San Francisco should get rain tomorrow:
Weekly Forecast
TUES WED THU FRI SAT SUN
14° 16° 17° 17° 18° 18°
9° 7° 6° 6° 7° 8°

Dan Jurafsky

Types of Questions

innovate achieve lead

- Factoid questions
  - *Who wrote "The Universal Declaration of Human Rights"?*
  - *How many calories are there in two slices of apple pie?*
  - *What is the average age of the onset of autism?*
  - *Where is Apple Computer based?*
- Complex (narrative) questions:
  - *In children with an acute febrile illness, what is the efficacy of acetaminophen in reducing fever?*
  - *What do scholars think about Jefferson's position on dealing with pirates?*

Commercial systems: mainly factoid questions

innovate achieve lead

Where is the Louvre Museum located?	In Paris, France
What's the abbreviation for limited partnership?	L.P.
What are the names of Odin's ravens?	Huginn and Muninn
What currency is used in China?	The yuan
What kind of nuts are used in marzipan?	almonds
What instrument does Max Roach play?	drums
What is the telephone number for Stanford University?	650–723–2300

Journey of QA Systems

innovate achieve lead

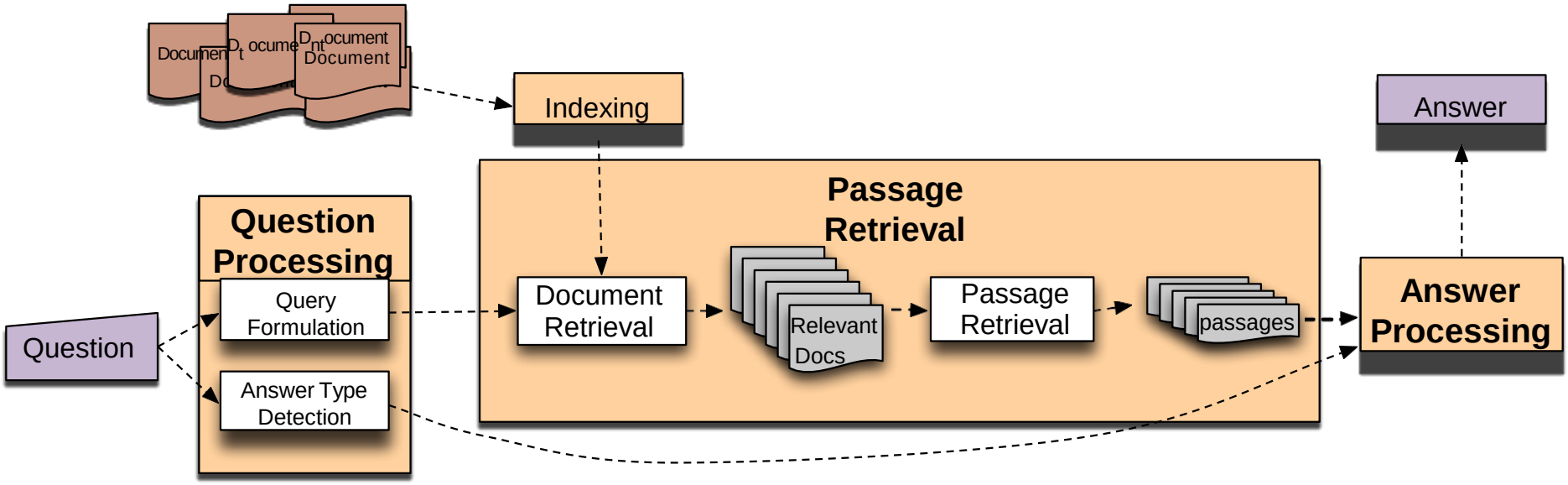
Evolution Path:

IR-based → Knowledge-based → Hybrid (IR + Knowledge) → Deep Learning → Transformers → RAG

IR- based:

- **Approach:** Retrieve relevant documents/passages from large text collections.
- **Techniques:** Keyword matching, TF-IDF, BM25.
- **Examples:** Early QA in search engines (AskJeeves, early Google).
- **Limitations:** No true understanding; returns documents, not direct answers.

IR--based Factoid QA



IR--based Factoid QA

- QUESTION PROCESSING
  - Detect question type, answer type, focus, relations
  - Formulate queries to send to a search engine
- PASSAGE RETRIEVAL
  - Retrieve ranked documents
  - Break into suitable passages and rerank
- ANSWER PROCESSING
  - Extract candidate answers
  - Rank candidates
    - using evidence from the text and external sources

Information Retrieval

IR systems retrieve relevant documents from a large corpus based on a query, using keyword matching, TF-IDF, BM25, or semantic search.

How It Works

1. Query is parsed and preprocessed
2. Index is searched (inverted index / embeddings)
3. Documents ranked by relevance score (BM25, cosine sim)
4. Top-k documents returned as answers

Tech Stack

- Elasticsearch / OpenSearch
- Apache Solr
- BM25 (rank_bm25 Python)
- FAISS / Annoy (vector search)
- Lucene, Whoosh

Real-World Use Cases

- Google/Bing web search
- Legal document retrieval
- Healthcare literature search (PubMed)

Knowledge--based approaches (Siri)

innovate achieve lead

- Build a semantic representation of the query
  - Times, dates, locations, entities, numeric quantities
- Map from this semantics to query structured data or resources
  - Geospatial databases
  - Ontologies (Wikipedia infoboxes, dbPedia, WordNet, Yago)
  - Restaurant review sources and reservation services
  - Scientific databases

Hybrid approaches (IBM Watson)

innovate achieve lead

- Build a shallow semantic representation of the query
- Generate answer candidates using IR methods
  - Augmented with ontologies and semi--structured data
- Score each candidate using richer knowledge sources
  - Geospatial databases
  - Temporal reasoning
  - Taxonomical classification

Question Answering: IBM's Watson

- Won Jeopardy on February 16, 2011!

WILLIAM WILKINSON'S
"AN ACCOUNT OF THE PRINCIPALITIES OF WALLACHIA AND MOLDOVIA"
INSPIRED THIS AUTHOR'S
MOST FAMOUS NOVEL

Bram Stoker

IBM Watson QA

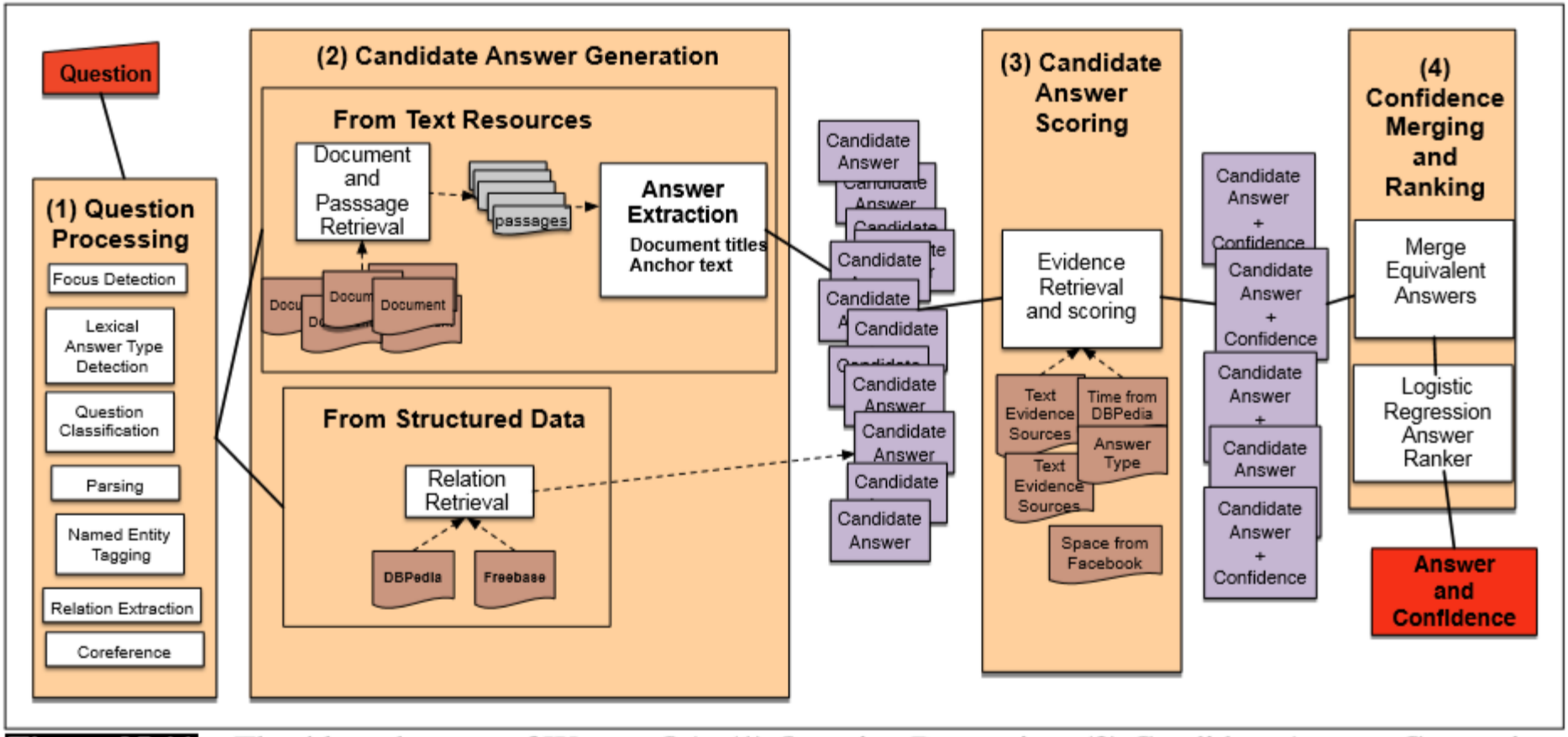


Figure 25.11 The 4 broad stages of Watson QA: (1) Question Processing, (2) Candidate Answer Generation, (3) Candidate Answer Scoring, and (4) Answer Merging and Confidence Scoring.

Deep Learning approach - QA

innovate achieve lead

- **Neural QA:**
CNNs, RNNs, and attention mechanisms learn semantic matching.

- **Transformers (BERT, RoBERTa):**
Contextual embeddings enable **Extractive QA** — selecting answer spans from text.
- *Example:* SQuAD dataset benchmarks

Generative QA: Models like GPT, T5 generate free-form answers.

Extractive QA

How it Works: The model is given a **Question** and a **Context** (a piece of text). It then "highlights" the span of text that contains the answer.

Analogy: A digital highlighter.

Example:

Context: "The Eiffel Tower was completed in 1889 and is located in Paris, France."

Question: "Where is the Eiffel Tower located?"

Answer: "Paris, France"

Pros: High accuracy, factually grounded, low risk of "hallucination."

Cons: Only works if the answer is *explicitly* in the text. Can't synthesize info.

QA using Deep Learning

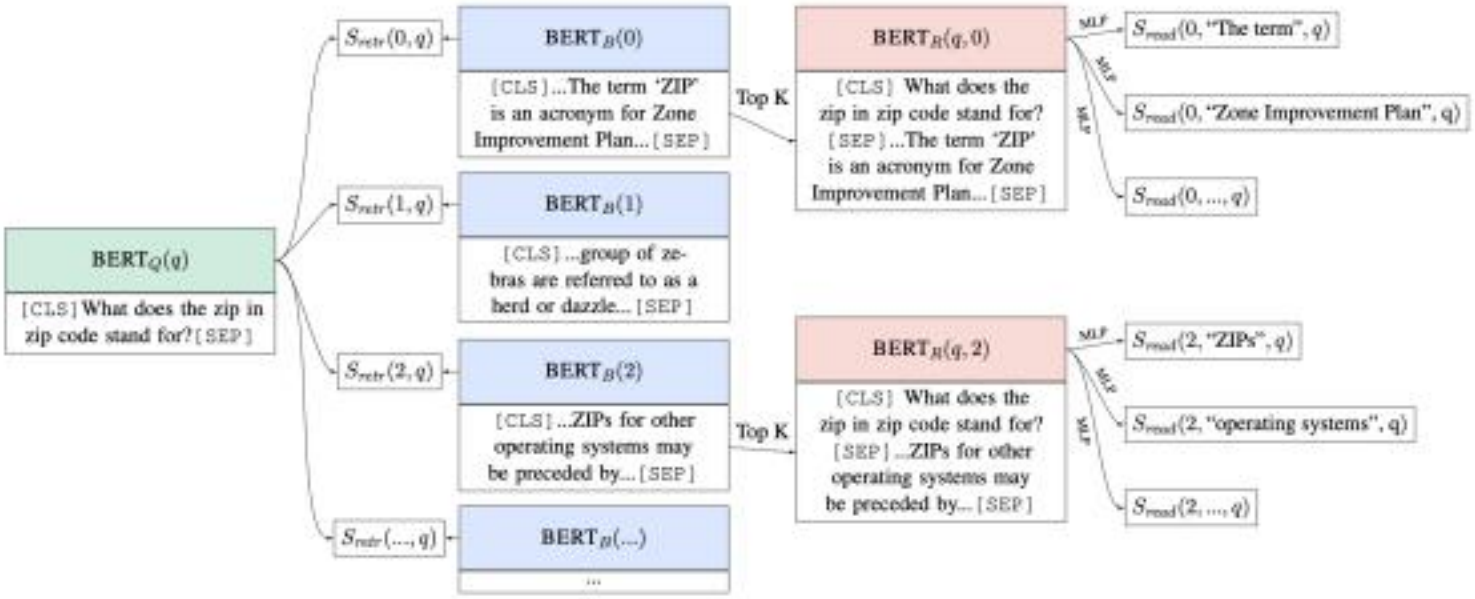


Image credit: (Lee et al., 2019)

Almost all the state-of-the-art question answering systems are built on top of end-to-end training and pre-trained language models (e.g., BERT)!

QA training datasets

Stanford question answering dataset (SQuAD)

- 100k annotated (passage, question, answer) triples

  Large-scale supervised datasets are also a key ingredient for training effective neural models for reading comprehension!

- Passages are selected from English Wikipedia, usually 100~150 words.
- Questions are crowd-sourced.
- Each answer is a short segment of text (or span) in the passage.

  This is a limitation— not all the questions can be answered in this way!

- SQuAD was for years the most popular reading comprehension dataset; it is "almost solved" today (though the underlying task is not,) and the state-of-the-art exceeds the estimated human performance.

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under gravity. The main forms of precipitation include drizzle, rain, sleet, snow, graupel and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals within a cloud. Short, intense periods of rain in scattered locations are called "showers".

What causes precipitation to fall?
gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?
graupel

Where do water droplets collide with ice crystals to form precipitation?
within a cloud

QA training datasets

innovate achieve lead

Other question answering datasets

- TriviaQA: Questions and answers by trivia enthusiasts. Independently collected web paragraphs that contain the answer and seem to discuss question, but no human verification that paragraph supports answer to question

- Natural Questions: Question drawn from frequently asked Google search questions. Answers from Wikipedia paragraphs. Answer can be substring, yes, no, or NOT_PRESENT. Verified by human annotation.

- HotpotQA. Constructed questions to be answered from the whole of Wikipedia which involve getting information from two pages to answer a multistep query:
  Q: Which novel by the author of "Armada" will be adapted as a feature film by Steven Spielberg? A: *Ready Player One*

QA using Deep Learning

How can we build a model to solve SQuAD?

(We are going to use **passage, paragraph and context,** as well as **question** and **query** interchangeably)

- Problem formulation
  - Input: $C = (c_1, c_2, \dots, c_N), Q = (q_1, q_2, \dots, q_M), c_i, q_i \in V$ N~100, M ~15
  - Output: $1 \leq \text{start} \leq \text{end} \leq N$ answer is a span in the passage

- A family of LSTM-based models with attention (2016–2018)

  Attentive Reader (Hermann et al., 2015), Stanford Attentive Reader (Chen et al., 2016), Match-LSTM (Wang et al., 2017), BiDAF (Seo et al., 2017), Dynamic coattention network (Xiong et al., 2017), DrQA (Chen et al., 2017), R-Net (Wang et al., 2017), ReasoNet (Shen et al., 2017)..

- Fine-tuning BERT-like models for reading comprehension (2019+)

QA in Deep Learning

- Transfer learning is the application of knowledge learned while solve one problem on other similar problems.
- Latest deep learning based word embedding's such as Bidirectional Encoder Representation from Transformers (BERT) *enable pre-trained Question Answer models trained on corpus from one domain to easily answer questions from another domain*.
- This makes is easy to introduce support for Question Answering in newer applications using pre-trained models.

QA Bot – NLP DeepLearning

- We will be implementing a chat bot that can answer questions based on a given story.
- **The bAbI project** by Facebook research
- https://research.fb.com/downloads/babi/

This page gather resources related to the bAbI project of Facebook AI Research which is organized towards the goal of automatic text understanding and reasoning.

QA Bot

- Story
  - Jane went to the store. Mike ran to the bedroom.
- Question
  - Is Mike in the store?
- Answer
  - No

QA Bot

End-to-End Memory Networks

- Sainbayar Sukhbaatar
- Arthur Szlam
- Jason Weston
- Rob Fergus

You must read the paper to understand this network!

1503.08895.pdf (arxiv.org)

QA Bot

- Model takes a discrete set of inputs **x1, ..., xn** that are to be stored in the memory, a query **q**, and outputs an answer **a**
- Each of the **x** , **q**, and **a** contains symbols coming from a dictionary with **V** words.
- The model writes all **x** to the memory up to a fixed buffer size, and then finds a continuous representation for the **x** and **q**.

QA Bot- NN Demo

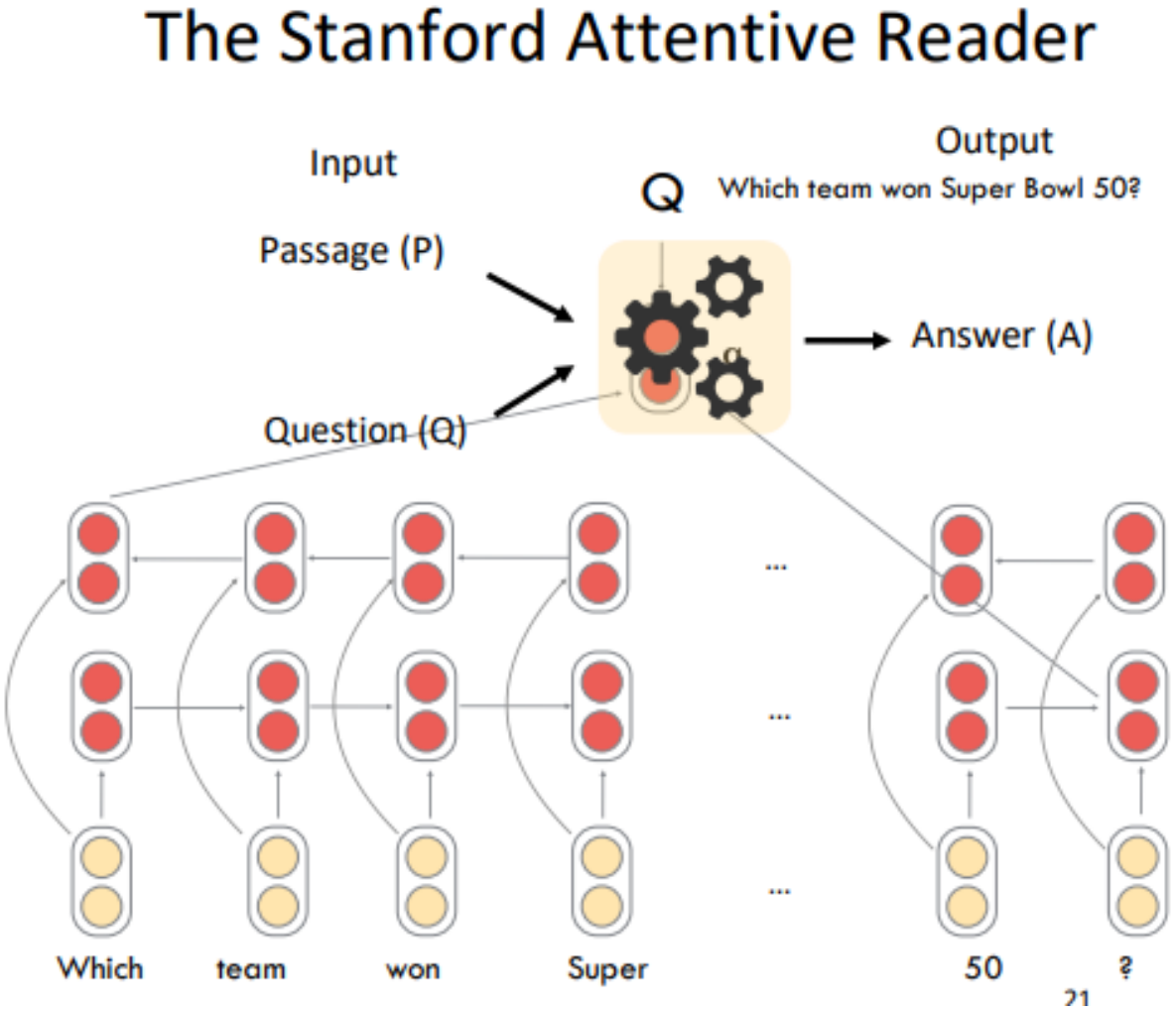
- Training data is a list of tuples.
- Each tuple has a Story, Question and an Answer.
- Training and Testing Data should be a part of Vocabulary.

- Padding: Pad_sequences

```
sample_text_1="bitty bought a bit of butter"
sample_text_2="but the bit of butter was a bit bitter"
sample_text_3="so she bought some better butter to make the bitter butter bette
r"
```

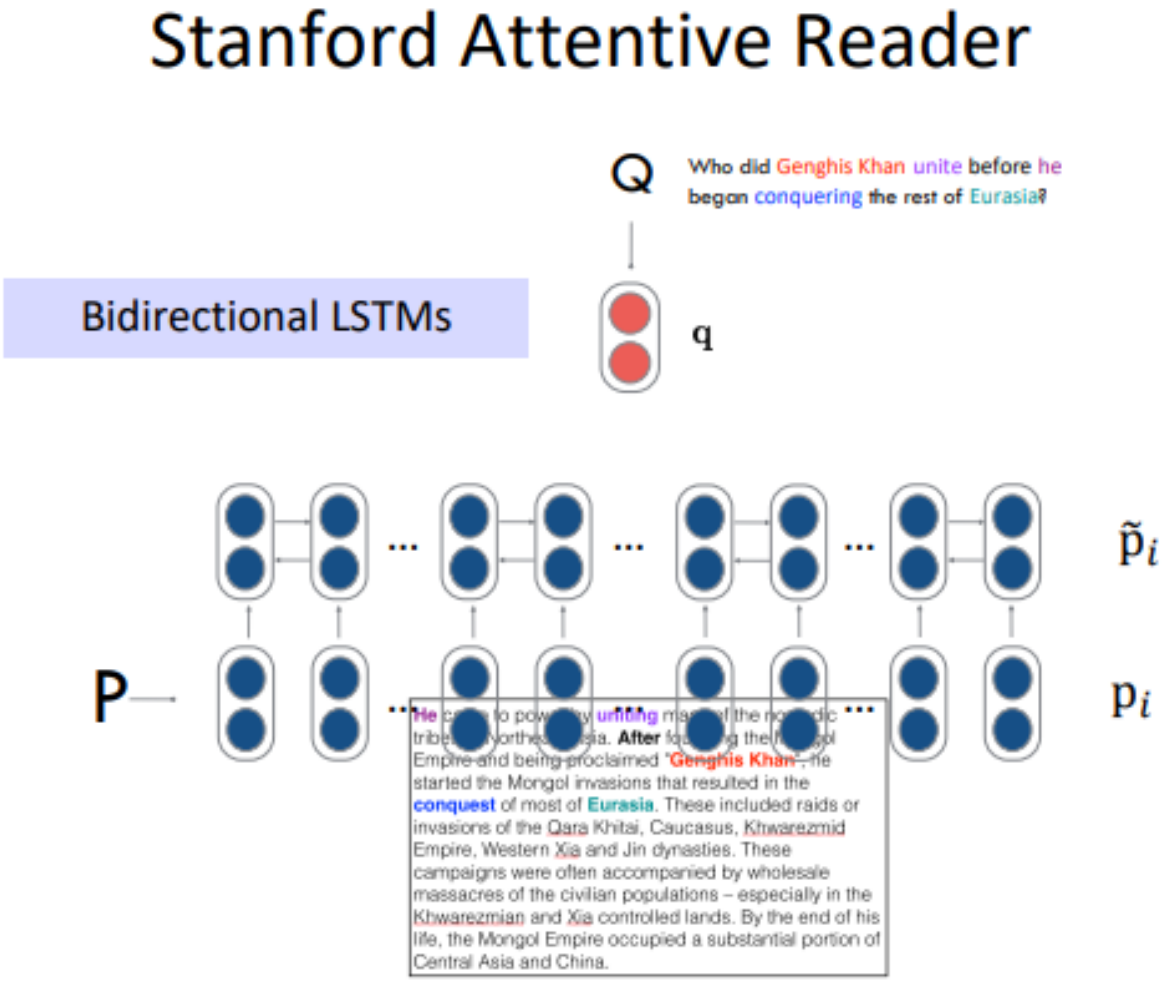
```
 The encoding for document 1  is :  [45, 16, 32, 27, 34, 33]
 The encoding for document 2  is :  [24, 2, 27, 34, 33, 37, 32, 27, 3]
 The encoding for document 3  is :  [22, 27, 16, 28, 35, 33, 7, 2, 2, 3, 33, 3
 5]

The padded encoding for document 1  is :  [45 16 32 27 34 33  0  0  0  0  0
0]
The padded encoding for document 2  is :  [24  2 27 34 33 37 32 27  3  0  0
0]
The padded encoding for document 3  is :  [22 27 16 28 35 33  7  2  2  3 33 3
5]
```

Word embedding of Question



(Use RNN in both directions - BiLSTM)

Word embedding of passage



Deep Learning architecture for QA

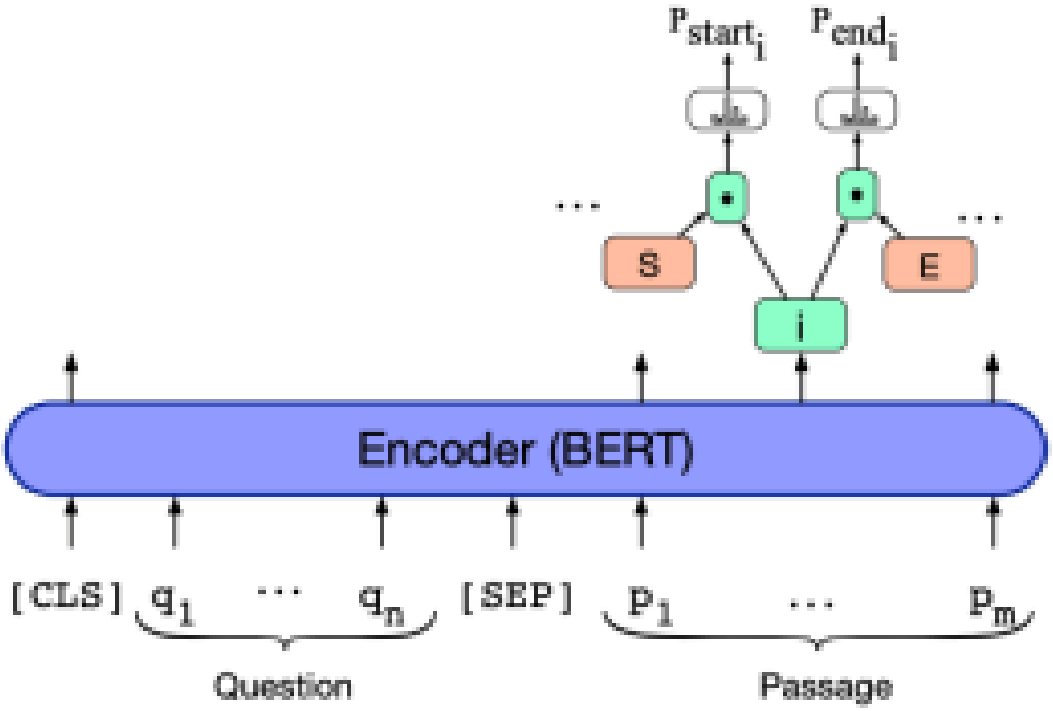
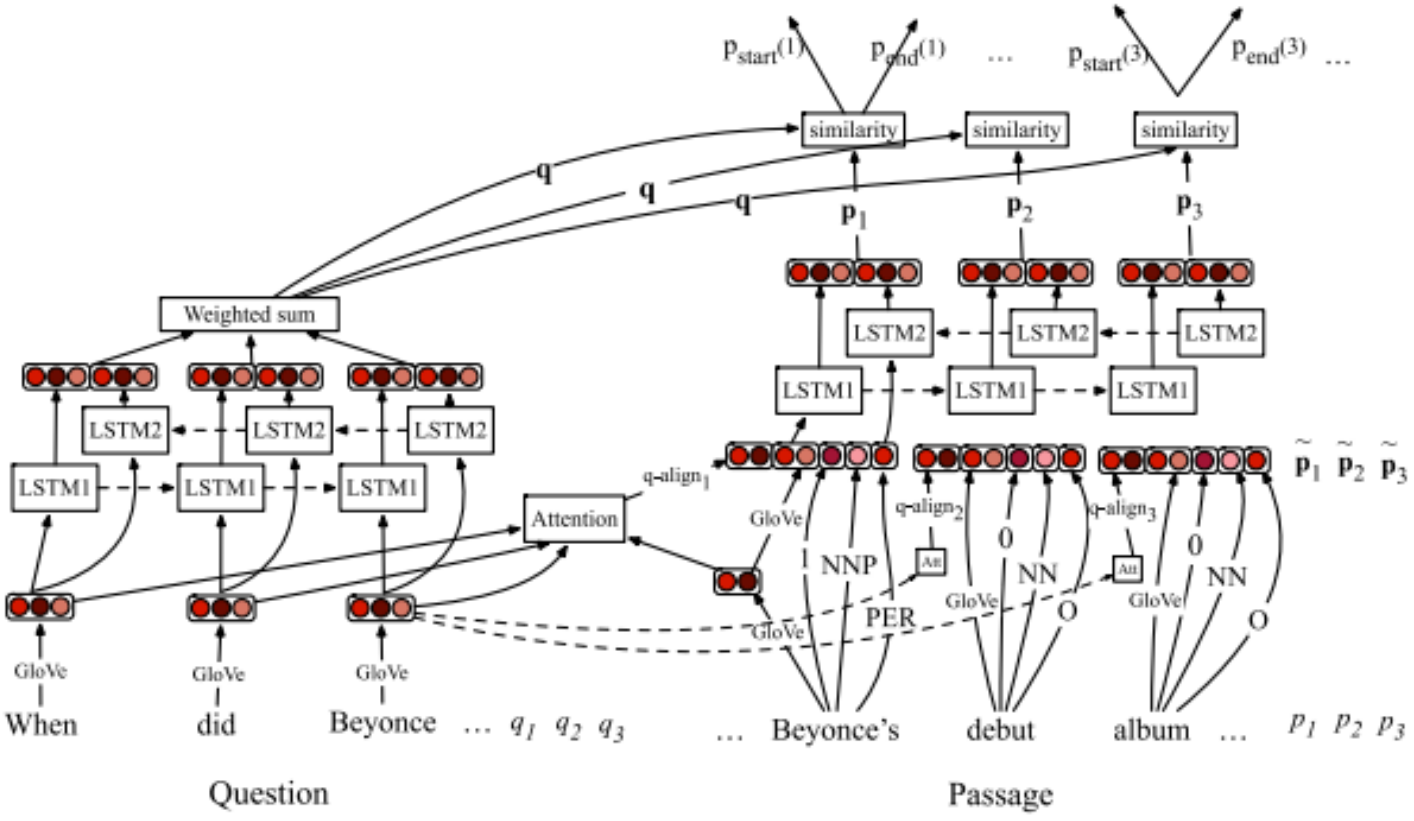


Image credit: J & M, edition 3

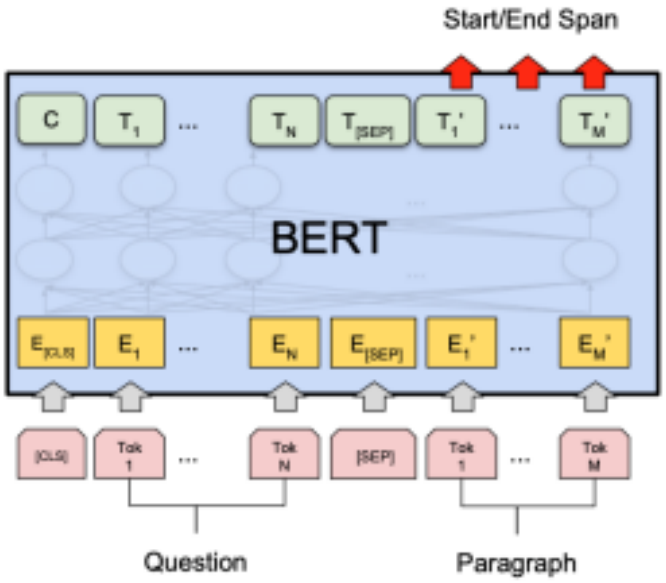
Deep Learning architecture for QA



Training objective: $\mathcal{L} = -\sum \log P^{(\text{start})}(a_{\text{start}}) - \sum \log P^{(\text{end})}(a_{\text{end}})$

Deep Learning architecture for QA

BERT for reading comprehension



$$\mathcal{L} = -\log p_{\text{start}}(s^*) - \log p_{\text{end}}(e^*)$$

$$p_{\text{start}}(i) = \text{softmax}_i(\mathbf{w}_{\text{start}}^\intercal \mathbf{h}_i)$$

$$p_{\text{end}}(i) = \text{softmax}_i(\mathbf{w}_{\text{end}}^\intercal \mathbf{h}_i)$$

where $\mathbf{h}_i$ is the hidden vector of c_i, returned by BERT

Question = Segment A

Passage = Segment B

Answer = predicting two endpoints in segment B

Question: How many parameters does BERT-large have?

Reference Text: BERT-large is really big... it has 24 layers and an embedding size of 1,024, for a total of 340M parameters! Altogether it is 1.34GB, so expect it to take a couple minutes to download to your Colab instance.

Image credit: https://mccormickml.com/

Open Domain QA

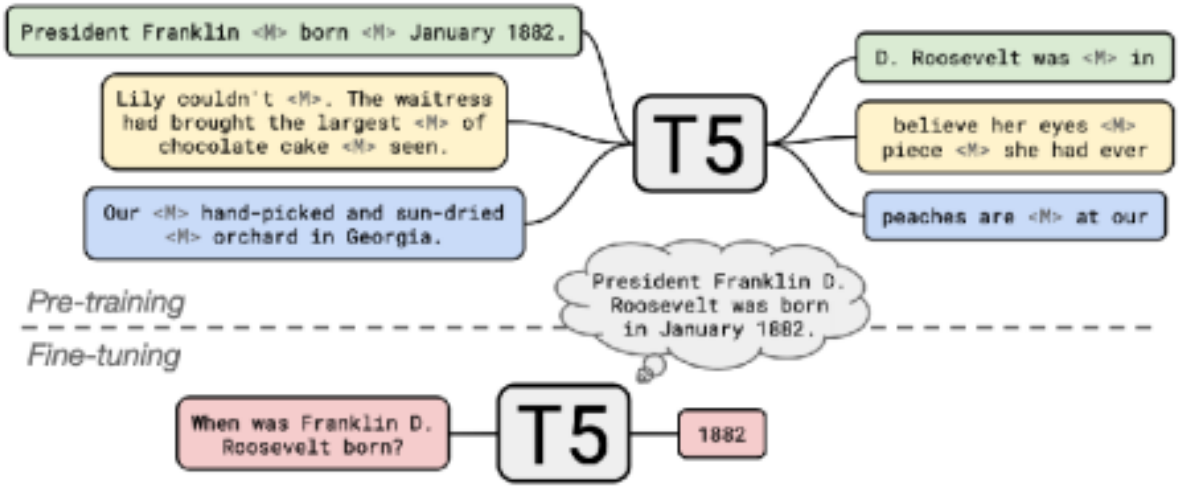
Open-domain question answering

- Different from reading comprehension, we don't assume a given passage.
- Instead, we only have access to a large collection of documents (e.g., Wikipedia). We don't know where the answer is located, and the goal is to return the answer for any open-domain questions.
- Much more challenging and a more practical problem!

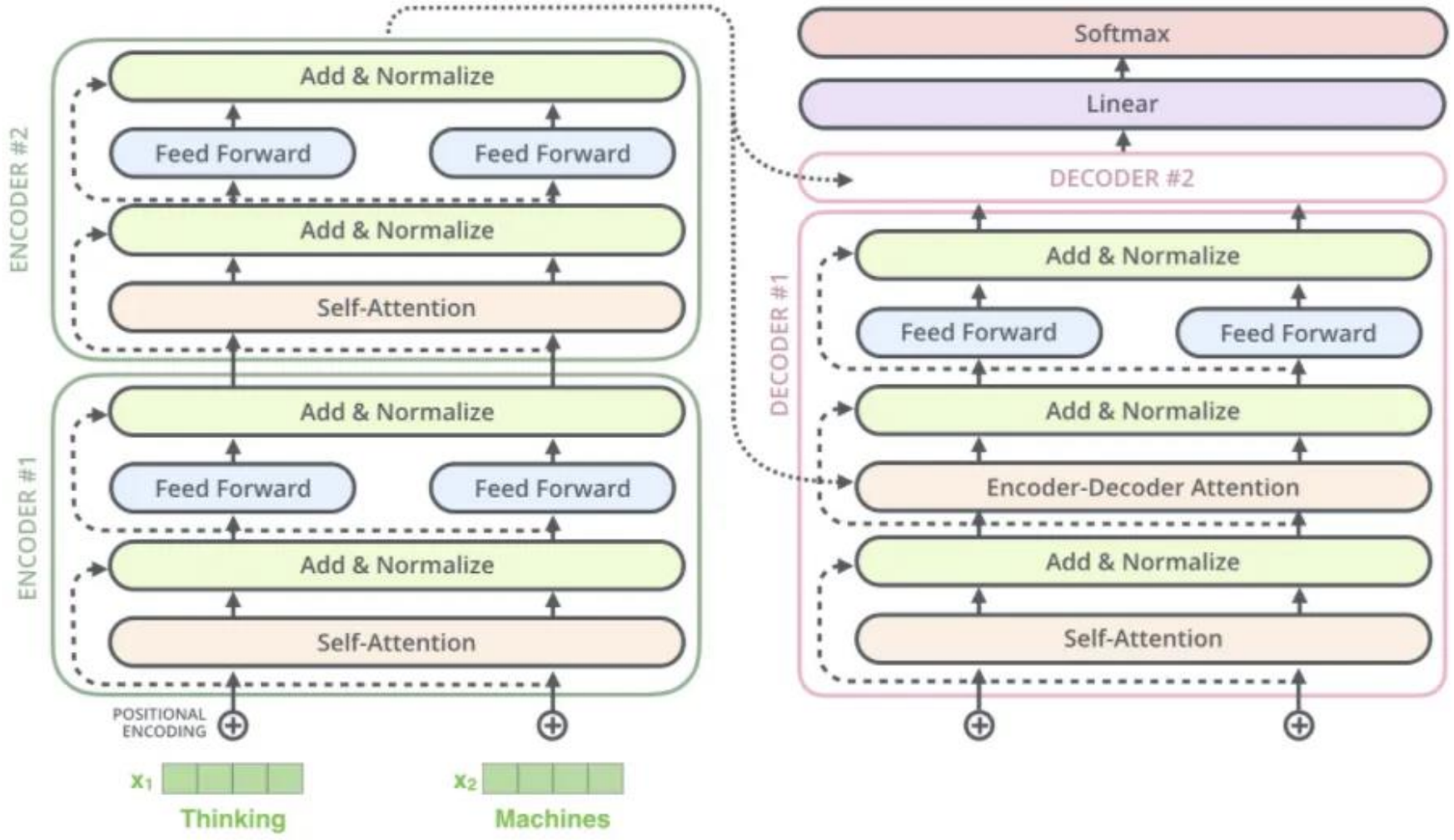
T5 LLM for QA

Large language models can do open-domain QA well

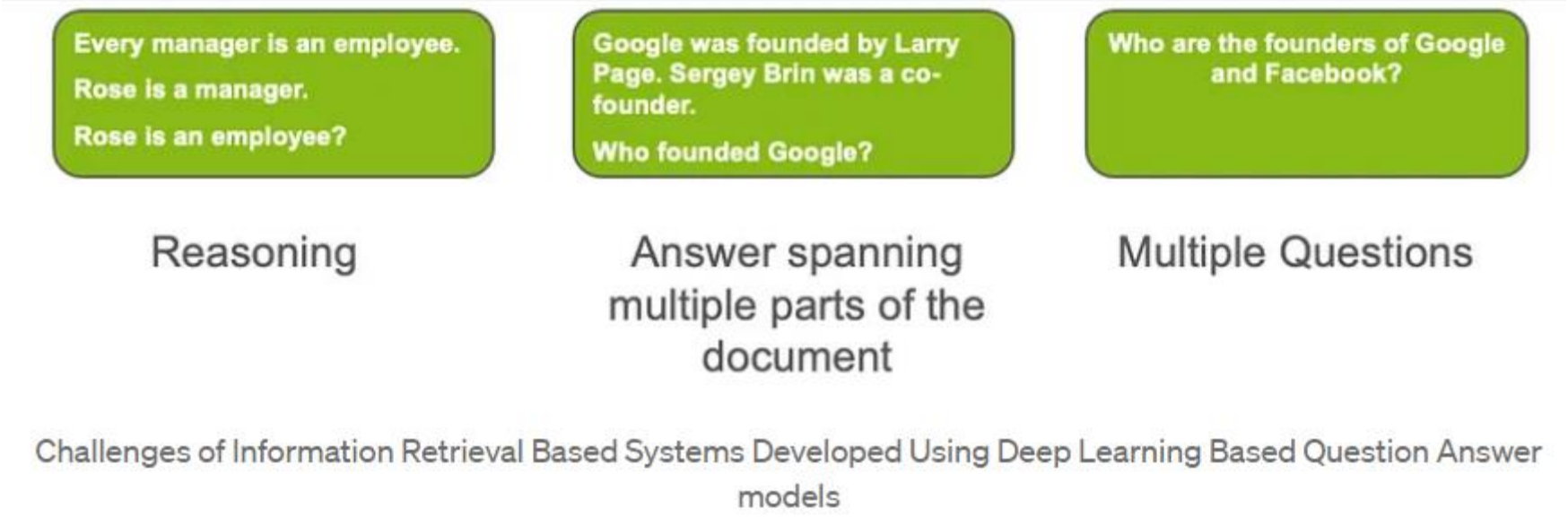
- … without an explicit retriever stage



T5 model Architecture



Challenges of Deep learning approaches



Challenges of Information Retrieval Based Systems Developed Using Deep Learning Based Question Answer models

Challenges of Deep learning approaches

- Incapable of answering questions that require reasoning.
- Deep learning based models for Question Answering take the input passage and questions and output the start and end position in the passage that contains the answer.
- Consequently they can not answer questions whose answer is spread across the document.
- Can not answer questions that have multiple sub questions whose answers are spread across the document.

QA Modern Approaches

- **Generative QA:** *Generates* a new answer from scratch.
- **Retrieval-Augmented Generation (RAG):** The modern hybrid that gets the best of both worlds.

Generative QA

How it Works: The model (a Large Language Model like GPT-4) uses its vast, pre-trained knowledge to generate a new, human-like answer.

Analogy: A subject-matter expert answering from memory.

Example:

- **Question:** "Why is the Eiffel Tower significant?"
- **Answer:** "The Eiffel Tower is significant because it was an architectural marvel for its time and has become an iconic symbol of Paris and French culture."

Pros: Very flexible, can answer "why" and "how" questions, highly conversational.

Cons: "Hallucinations" (making up facts) and **stale data** (its knowledge is frozen in time).

Generative AI for Question Answering

LLMs generate free-form natural language answers using parametric knowledge learned during training — no external retrieval required at inference time.

Key Techniques

Prompt Engineering

Zero-shot, few-shot, CoT prompting

Fine-tuning

Task-specific SFT on QA datasets (SQuAD, TriviaQA)

Instruction Tuning

RLHF / DPO alignment for factual responses

In-Context Learning

Provide examples in the prompt context window

Tech Stack

- OpenAI GPT-4 / GPT-4o
- Google Gemini 1.5 Pro
- Anthropic Claude 3.5 / 3.7
- Meta LLaMA 3.x (open-source)
- Mistral / Mixtral
- Hugging Face Transformers

Real-World Use Cases

- ChatGPT Q&A / customer support
- Code assistant (GitHub Copilot)

The Big Problem: Hallucinations & Stale Data

You can't trust a standard Generative LLM with your private company data.

- **It's a "Black Box":** You don't know *why* it gave an answer (no citations).
- **It's Stale:** It has no idea your new product (Project X) launched last week.
- **It's Insecure:** It wasn't trained on your *private* documents.

This leads to the most important approach used in business today...

Retrieval Based Methods

- Modern large language models (LLMs) — like ChatGPT, Gemini, or Claude — use **retrieval-based methods** behind the scenes.
- Instead of generating answers purely from memory, they:
- **Retrieve** relevant text chunks from a large document database (IR stage).
- **Generate** the final answer using a language model conditioned on those documents.
- This is called **Retrieval-Augmented Generation (RAG)** or **retrieval-augmented QA**.

Retrieval Based Methods

- Example:
- User: “Who discovered penicillin?”
- System: retrieves passages mentioning “Alexander Fleming” → summarizes →
  Answer: “Penicillin was discovered by Alexander Fleming in 1928.”
- So — modern IR-based QA = **search + LLM reasoning**.

Retrieval-Augmented Generation

The "Best of Both Worlds" Solution.

- RAG combines a **Retriever** (like a search engine) with a **Generator** (an LLM).
- **Analogy:** An "open-book" exam. The LLM doesn't answer from memory. It is *given* the relevant documents (the "book") and told, "Use *only* these documents to answer the question."
- This solves the hallucination and stale data problem.

RAG Pipeline

- **Query:** User asks, "What is our company's WFH policy?"
- **Retrieve:** The system *first* searches a private database (a **Vector Database**) of all company HR documents. It finds the 3 most relevant docs.
- **Augment:** The system builds a new prompt:
  - "User Question: What is our WFH policy?"
  - "Context: [Here is the text of the 'Work-From-Home Policy' doc...]"
- **Generate:** This prompt is fed to the LLM. The LLM then generates an answer *based only on the provided context*.
  - **Answer:** "Our WFH policy allows employees to work from home 2 days per week..."

RAG Tech stack

innovate achieve lead

The Brains (Models): LLMs

The Memory (Data): Vector Databases

The Glue (Orchestration): Frameworks

The Interface (App): Frontend/Backend

RAG Tech stack

Large Language Models (LLMs): These generate the final answer.

- **Closed Source:** OpenAI (GPT-4, GPT-4o), Anthropic (Claude 3), Google (Gemini).
- **Open Source:** Meta (Llama 3), Mistral (Mixtral).

Embedding Models: These are crucial for the "Retrieval" step. They turn text documents into numbers (vectors) so we can find *semantically similar* documents.

Examples: all-MiniLM-L6-v2, text-embedding-3-small

Vector Datastore

innovate achieve lead

You can't find "semantically similar" documents in a traditional SQL database.

Vector Datastore are purpose-built to store and query these "embedding" vectors at massive scale.

How it works: It finds documents that are *conceptually related* to the query, not just ones that share keywords.

Query: "rules for leave"

Finds: "policy for taking time off"

Examples: Pinecone, ChromaDB, FAISS, Weaviate, Milvus

RAG Orchestration

These frameworks manage the entire RAG pipeline (Query -> Retrieve -> Augment -> Generate). They are the "plumbing" that connects everything.

LangChain: The most popular and comprehensive library. A "Swiss Army Knife" for building complex LLM applications (agents, chains).

LlamaIndex: A simpler, data-focused framework that excels at *one thing*: building powerful RAG pipelines. It's built for "ingesting, indexing, and querying" your data.

RAG

RAG combines retrieval of relevant documents with generative LLMs — grounding answers in external, up-to-date knowledge to reduce hallucinations.

RAG Pipeline

Tech Stack

Orchestration: LangChain, LlamaIndex, Haystack

Vector DBs: Pinecone, Weaviate, Qdrant, Chroma, pgvector

Embeddings: OpenAI text-embedding-3, Sentence-Transformers

LLMs: GPT-4o, Claude 3.5, Gemini 1.5

Reranking: Cohere Rerank, Cross-Encoder (SBERT)

RAG Variants

- Naive RAG — basic retrieve + generate
- Advanced RAG — re-ranking, HyDE, query expansion
- Modular RAG — routing, fusion, iterative

Real-World Use Cases

- Enterprise knowledge bases (Confluence, SharePoint)
- Legal & financial document QA
- Customer support with product docs

Comparison of QA approaches

Feature	Extractive QA	Generative QA (LLM)	RAG
Answer Source	Exact text span	Model's internal memory	External, current documents
Hallucination Risk	Very Low	High	Low (mitigated)
Data Freshness	Depends on context	Stale (frozen in time)	Real-time (just update docs)
Citations?	Yes (the source doc)	No	Yes (can cite sources)
Best For	Fact-checking	Creative chat	Enterprise QA, Chatbots

Case Study: "AskHR" - An Internal Enterprise Chatbot

- Company: A 10,000-person global tech firm.
- **The Problem**: The HR department is overwhelmed with thousands of repetitive employee questions every month:
  - "How much PTO do I have?"
  - "What's the policy for travel expenses?"
  - "Where do I find my tax forms?"
- These answers exist but are buried in a 500-page PDF on an internal site

- **Goal:** Build a 24/7 chatbot that can answer 90% of HR questions instantly and accurately, citing its sources.
- **The Choice: RAG**
- **Why not Generative?** A base LLM (like ChatGPT) knows *nothing* about the company's specific policies and would make up answers (hallucinate).
- **Why not Extractive?** The policies are complex and require synthesis, not just "highlighting" a single sentence

Techstack for AskHR

innovate achieve lead

Data Ingestion (Offline):

- All 200+ HR documents (PDFs, Word docs, site pages) are chunked.
- An **Embedding Model** converts these chunks into vectors.
- All vectors are loaded into a **Vector Database (Pinecone)**.

QA Application (Real-time):

- An employee asks, "Can I use my corporate card for conference tickets?"
- **LangChain** (the "glue") takes the query.
- It queries **Pinecone** to find the top 3 relevant doc chunks (e.g., the "Travel Policy" and "Corporate Card" docs).
- It passes the query + chunks to the **LLM (Claude 3)**.

The LLM generates the answer: "Yes, you can use your corporate card for conference tickets, but you must get manager approval first (as per the Travel Policy, pg. 4)."

Agentic AI for Question Answering

Agentic AI uses LLMs as reasoning engines that autonomously plan, use tools (search, code execution, APIs), and iteratively refine answers for complex multi-step queries.

Agentic Loop (ReAct Pattern)

Key Capabilities

Multi-step reasoning: Breaks complex questions into sub-tasks

Tool use: Web search, code execution, DB queries, APIs

Memory: Short-term (context) + long-term (vector store)

Planning: Task decomposition (ToT, LATS, Plan-and-Execute)

Tech Stack

- LangChain Agents / LangGraph
- AutoGen (Microsoft)
- CrewAI
- OpenAI Function Calling / Assistants API
- Semantic Kernel (Microsoft)
- smolagents (Hugging Face)

Real-World Use Cases

- Research assistant (Deep Research)
- Software dev agent (Devin, Claude Code)
- Financial analysis agents

Comparison of approaches

Dimension	IR	Generative AI	RAG	Agentic AI
Answer Type	Document snippet	Free-form text	Grounded text	Multi-step answer
Knowledge Source	External corpus	Parametric (trained)	External + LLM	Tools + LLM + Memory
Hallucination Risk	Low	High	Medium-Low	Medium
Latency	Very low	Low	Medium	High
Accuracy (complex Qs)	Low	Medium	High	Very High
Requires Internet?	No	No	Optional	Often Yes
Cost	Low	Medium	Medium-High	High

Evaluation Measures

Metrics are categorized by what they measure in QA systems:

Retrieval Metrics

- Precision@k
- Recall@k
- MRR (Mean Reciprocal Rank)
- MAP (Mean Avg. Precision)
- nDCG

Answer Quality Metrics

- Exact Match (EM)
- F1 Score
- ROUGE-L
- BLEU
- BERTScore

LLM-based Metrics

- Faithfulness
- Answer Relevance
- Context Precision
- Context Recall
- Hallucination Rate

End-to-End Metrics

- Human Eval (Likert)
- Correctness
- Groundedness
- Completeness
- Conciseness

Precision, Recall & F1 Score

Precision@k

How many of the top-k retrieved results are relevant?

```
Formula:  Precision@k = (# Relevant docs in top-k) / k
```

Ex: Top-5 results: 3 relevant → Precision@5 = 3/5 = 0.60

Recall@k

What fraction of all relevant docs were found in top-k?

```
Formula:  Recall@k = (# Relevant docs in top-k) / (Total relevant docs)
```

Ex: Top-5 results: 3 relevant, total relevant = 8 → Recall@5 = 3/8 = 0.375

F1 Score

Harmonic mean of Precision and Recall; balances both.

```
Formula:  F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Ex: Precision=0.60, Recall=0.375 → F1 = 2×(0.60×0.375)/(0.60+0.375) = 0.46

MRR, MAP & nDCG

MRR — Mean Reciprocal Rank

Average of reciprocal ranks of the first relevant document per query.

Formula: $MRR = (1/|Q|) \times \Sigma\ (1\ /\ rank\_i)$

Ex: Query 1: first relevant at rank 2 → 1/2; Query 2: rank 1 → 1/1
MRR = (0.5 + 1.0) / 2 = 0.75

MAP — Mean Average Precision

Mean of Average Precision across all queries; penalizes irrelevant results.

Formula: $MAP = (1/|Q|) \times \Sigma\ AveP(q)$ where $AveP = \Sigma\ (P@k)/\ R$

Ex: Relevant at ranks 1,3,5 (R=3): AveP = (1/1 + 2/3 + 3/5) / 3 = 0.756

nDCG — Normalized Discounted Cumulative Gain

Considers graded relevance; discounts gains for lower-ranked results. Normalized to [0,1].

Formula: $nDCG@k = DCG@k\ /\ IDCG@k$ where $DCG@k = \Sigma\ rel_i\ /\ \log_2(i+1)$

Ex: Gains [3,2,0,1]: DCG = 3/1 + 2/1.58 + 0 + 1/2.32 = 4.695; nDCG = 4.695 / IDCG

Common Evaluation Metrics

1. *Accuracy* (does answer match gold--labeled answer?)
2. *Mean Reciprocal Rank*
   - For each query return a ranked list of M candidate answers.
   - Query score is 1/Rank of the first correct answer
     - *If first answer is correct: 1*
     - *else if second answer is correct: ½*
     - *else if third answer is correct: ⅓, etc.*
     - *Score is 0 if none of the M answers are correct*
   - Take the mean over all N queries

$$MRR = \frac{\sum_{i=1}^{N} \frac{1}{rank_i}}{N}$$

Exact Match, ROUGE-L & BERTScore

Exact Match (EM)

Binary — 1 only if predicted answer exactly matches the ground truth (after normalization).

Formula: $EM = 1 \text{ if } normalize(pred) == normalize(gold) \text{ else } 0$

Ex: Gold: 'Albert Einstein' Pred: 'albert einstein' → EM = 1 (after lowercasing)

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation)

Based on Longest Common Subsequence (LCS); captures sentence-level structure similarity.

Formula: $ROUGE\text{-}L = F_1(LCS) = 2 \times P_{lcs} \times R_{lcs} / (P_{lcs} + R_{lcs})$

Ex: Ref: 'cat sat on mat' | Pred: 'cat on mat' → LCS=3, R=3/4=0.75, P=3/3=1.0 → ROUGE-L=0.857

BERTScore

Uses contextual BERT embeddings. Computes cosine similarity between tokens for semantic match.

Formula: $BERTScore\ F_1 = 2 \times Precision_{BERT} \times Recall_{BERT} / (P + R)$

Ex: Ref: 'He was the German physicist' Pred: 'German scientist' → BERTScore ≈ 0.92 (semantic match)

RAGAS & LLM-as-Judge Metrics

RAGAS (Retrieval Augmented Generation Assessment) provides automated, reference-free evaluation for RAG pipelines using an LLM-as-Judge.

Faithfulness

Measures whether answer is grounded in retrieved context (no hallucination).

```
= Claims supported by context / Total claims in answer
```

Ex: Answer has 5 claims, 4 supported by context → Faithfulness = 0.80

Answer Relevance

Generates questions from the answer, checks if they match the original query.

```
= avg cosine_sim(generated_questions, original_question)
```

Ex: High score means the answer actually addresses the question asked.

Context Precision

Checks if the retriever fetched only relevant documents (precision of retrieval).

```
= Relevant chunks in context / Total chunks retrieved
```

Ex: 3 of 5 retrieved chunks are relevant → Context Precision = 0.60

Context Recall

Measures if the context contains enough info to answer the question fully.

```
= Claims in ground truth supported by context / Total GT claims
```

Ex: Ground truth has 4 claims, 3 found in context → Context Recall = 0.75

Summary

IR	Information Retrieval excels at speed & scalability — ideal when exact document lookup suffices (search engines, e-discovery).
Gen	Generative AI produces fluent answers from parametric knowledge — but risks hallucination without grounding or verification.
RAG	RAG is today's production sweet spot — combining retrieval grounding with LLM fluency. Dominant in enterprise knowledge QA.
Agent	Agentic AI handles complex, multi-step queries requiring tool use and reasoning chains — highest capability but also highest cost & latency.
Eval	Evaluation must cover retrieval (nDCG, MAP), answer quality (ROUGE-L, BERTScore), and LLM-based metrics (RAGAS Faithfulness, Relevance).

References

- Speech and Language processing: An introduction to Natural Language Processing, Computational Linguistics and speech Recognition by Daniel Jurafsky and James H. Martin[3rd edition] - Chapter 25
- https://medium.com/@aakashgoel12/question-answering-system-on-corona-approach-01-6ef9799695cb
- https://www.machinelearningplus.com/nlp/chatbot-with-rasa-and-spacy/
- https://analyticsindiamag.com/10-question-answering-datasets-to-build-robust-chatbot-systems/
- https://github.com/ElizaLo/Question-Answering-based-on-SQuAD
- https://intersog.com/blog/the-basics-of-qa-systems-from-a-single-function-to-a-pre-trained-nlp-model-using-python/
- https://paperswithcode.com/task/question-answering/latest
- https://www.youtube.com/watch?v=NcqfHa0_YmU
- https://www.youtube.com/watch?v=3XiJrn_8F9Q
- **RAGAS: How to Evaluate a RAG Application Like a Pro for Beginners**

Code references

- https://colab.research.google.com/github/shahules786/openai-cookbook/blob/ragas/examples/evaluation/ragas/openai-ragas-eval-cookbook.ipynb
- https://colab.research.google.com/github/langfuse/langfuse-docs/blob/main/cookbook/evaluation_of_rag_with_ragas.ipynb

Research Papers

KERAG: Knowledge-Enhanced RAG	Integrates a broad subgraph from a knowledge graph + LLM chain-of-thought to improve answer coverage and reduce hallucination.	ACL Anthology
BYOKG-RAG: Multi-Strategy Graph Retrieval for KGQA	Combines LLM-generated artifacts with graph tools to iteratively retrieve from custom KGs, improving robustness across varied graphs.	ACL Anthology

Research Papers- Self read

D-RAG: Differentiable RAG for KGQA	Jointly optimizes retriever and generator via differentiable subgraph sampling to boost performance on KGQA.	ACL Anthology
Enhancing Complex Reasoning in KGQA (Aqua-QA)	Approximates query graphs from natural language to handle incomplete KGs and logical reasoning.	ACL Anthology
RGR-KBQA: Generating Logical Forms with KG-Enhanced LLMs	Uses a retrieve-generate-retrieve pipeline to improve logical form generation and reduce hallucination in KBQA.	ACL Anthology

Research Papers- Self read

Knowledge Graph-Extended RAG (KG-RAG)	Integrates LLMs with KGs using in-context learning + chain-of-thought for explainable multi-hop QA.	EmergentMind
LLMs + Knowledge Graphs for QA: Survey	Categorizes and analyzes methods of combining LLMs + KGs for QA, highlighting future directions.	ACL Anthology
MDKAG: Multimodal KG for Educational QA	Builds a QA system using a multimodal discipline-specific knowledge graph for educational questions.	MDPI
RAGulating Compliance: Multi-Agent KG QA	Uses a multi-agent system and a dynamically maintained KG to answer regulatory compliance questions with traceability.	arXiv

Research Papers- Self read

StepChain GraphRAG: Multi-Hop QA	Decomposes questions and builds evidence chains via BFS traversal on dynamically constructed subgraphs for interpretable multi-hop QA.	arXiv
Interpretable QA with Knowledge Graphs	Answers via KG retrieval without LLMs, using paraphrasing of KG edge relations for human-readable output.	arXiv
AutoGraph-R1: RL for KG Construction for QA	Uses reinforcement learning to build KGs optimized for downstream QA performance.	arXiv